

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1714

COURSE OUTCOME COVERED

CO5: Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.

(BTL 5)

Assessment Tool Weightage: 20%

Duration : 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

I _G Charan Teja 192372301_ (Name / Reg No) certify that this submission is my original work and that I

have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Reverse Engineering a Healthcare Diagnosis Assistant

1. Introduction

A Healthcare Diagnosis Assistant is an AI-based system that analyzes large volumes of patient information such as vital signs, laboratory results, medical history, and clinical observations. Its main purpose is to identify high-risk patients and provide alerts for immediate physician review. Since hospital data is generated continuously and may come from many departments, the system must process information in a scalable and fault-tolerant manner. By observing the inputs and outputs of such a system, the likely architecture can be reverse engineered as a distributed pipeline using PySpark, RDD operations, machine learning models, and intelligent agents.

2. Reconstructed System Architecture

The system can be divided into five major layers. The data ingestion layer collects patient records from electronic health records, laboratory systems, ICU monitors, and wearable devices. These records are stored in distributed storage. The PySpark processing layer converts incoming data into RDDs and distributes them across worker nodes. The feature extraction layer converts raw medical records into useful features such as heart rate, blood pressure, oxygen level, glucose level, abnormal laboratory values, and recent changes in vital signs. The AI risk-scoring layer uses these features to predict the probability that a patient requires urgent attention. Finally, an intelligent alert layer sends high-priority cases to physicians and clinical staff.

3. PySpark and RDD-Based Data Pipeline

PySpark supports parallel processing of large healthcare datasets. Each patient record can be represented as an RDD element and processed across multiple nodes. The `map()` transformation can clean records and extract features from each patient's data. `filter()` can remove incomplete or invalid records and also select patients with abnormal measurements. `reduceByKey()` can aggregate repeated observations using a patient identifier, while `join()` can combine vital signs with laboratory results and medical history. Cached RDDs can be reused by different models without repeated computation. RDD lineage and fault tolerance allow lost partitions to be recomputed, improving reliability for a distributed healthcare environment.

4. Feature Extraction from Patient Records

The feature extraction process transforms raw medical data into structured information for machine learning. Important features may include age, temperature, heart rate, blood pressure, respiratory rate, oxygen saturation, glucose level, white blood cell count, and other laboratory measurements. Historical features such as previous diagnoses and trends in vital signs can also be included. For example, a sudden fall in oxygen saturation may be more important than a single normal reading. PySpark workers extract these features in parallel, normalize the values, handle missing data, and create a feature vector for every patient. The resulting data is then supplied to the risk prediction model.

5. Risk Scoring and Decision Making

The risk-scoring component estimates how likely a patient is to require immediate medical attention. A machine learning classifier can analyze the extracted feature vector and generate a risk score between low and high risk. The decision engine can use thresholds to classify patients into categories such as normal, moderate risk, high risk, and critical. To reduce false alerts, the system may combine model predictions with rule-based checks, such as extremely abnormal vital signs or dangerous laboratory values. Scores can also be aggregated

over time so that a rapidly worsening patient receives a higher priority than a patient whose condition remains stable.

6. Intelligent Agent-Based Alert Mechanism

A group of intelligent agents can coordinate the diagnosis workflow. A data collection agent gathers records from different hospital systems. A preprocessing agent validates and cleans incoming data. A feature extraction agent generates the clinical features required by the AI model. A risk assessment agent calculates the patient's risk score. An alert agent checks the severity and sends notifications to the appropriate physician or emergency team. A monitoring agent tracks whether the alert has been acknowledged and can escalate the case if necessary. These agents communicate through shared distributed services or message queues, allowing each task to operate independently while contributing to a complete diagnosis pipeline.

7. Real-Time Processing Workflow

The likely workflow begins when new patient vitals or laboratory results are received. The data is converted into distributed records and processed using PySpark transformations. Features are extracted and combined with the patient's previous information. The AI model calculates a risk score, after which the intelligent agents decide whether an alert is required. Low-risk records are stored for future analysis, while high-risk records are placed at the top of the physician review queue. This workflow supports continuous monitoring and enables urgent cases to be identified quickly even when thousands of patient records are being processed at the same time.

8. Fault Tolerance and Scalability

The distributed design makes the system suitable for large hospitals and healthcare networks. RDDs provide fault tolerance through lineage, allowing failed data partitions to be recomputed. Data can be divided among multiple worker nodes, increasing processing capacity as the number of patients grows. Replicated storage and checkpointing can protect important intermediate results. The system can also monitor model performance and processing delays. If workload increases, additional computing nodes can be added to maintain acceptable response times. Therefore, PySpark and distributed RDD operations provide both scalability and reliability for the reconstructed healthcare AI system.

9. Conclusion

The Healthcare Diagnosis Assistant can be reverse engineered as a distributed AI system that collects patient data, processes it using PySpark RDD operations, extracts clinically useful features, and generates risk scores through machine learning. Intelligent agents coordinate data collection, preprocessing, risk assessment, alert generation, and escalation. RDD transformations such as `map()`, `filter()`, `reduceByKey()`, and `join()` support scalable processing and integration of different patient data sources. The final system helps prioritize high-risk cases for immediate physician review while maintaining fault tolerance and supporting real-time healthcare decision making.